

Zadanie 2 Zakładam, że wierzchołków jest n i są ponumerowane liczbami od 1 do n oraz korzeń ma numer 1. Przygotowanie wygląda następująco: tworzę 3 tablice długości n - `InOrder`, `InOrderInv`, `Depth`. Tablice `InOrder` tworzę przeglądając drzewo w porządku `InOrder`, a tablicę `Depth` tworzę licząc głębokość wierzchołka podczas przeglądania drzewa w tym porządku, w ten sposób aby na i -tej pozycji była głębokość i -tego wierzchołka w porządku `InOrder`. To wykonuje się w czasie $\Theta(n)$ i nie wymaga więcej pamięci niż $\Theta(n)$. Powstała tablica tworzy permutację $\{1, \dots, n\}$ i w `InOrderInv` wpisuje jej odwrotność następująco. Iterując się po każdym indeksie $i = 1, \dots, n$ wpisuje: `InOrderInv[InOrder[i]] = i`. To wykonuje się w czasie $\Theta(n)$. Co sprawia, że pamięciowo zużywamy $3n \in \Theta(n)$ oraz czasowo podobnie co najwyżej $3n \in \Theta(n)$. Zatem ta część przygotowania ma złożoność czasową $\Theta(n)$ i pamięciową $\Theta(n)$.

Następnie dzielę tablicę `Depth` na około $\sqrt{n}$ bloków długości dokładnie (oprócz być może ostatniego) $\lceil \sqrt{n} \rceil$. Dla każdego bloku liczę minimum przechodząc przez każdy element bloku otrzymując około $\sqrt{n}$ indeksów, które zapamiętuje potrzebując $\Theta(\sqrt{n})$ pamięci. Złożoność czasowa tej operacji to $\Theta(\sqrt{n}\sqrt{n}) = \Theta(n)$.

Zatem całe przygotowanie wykonują się w czasie $\Theta(n)$ i wymagają $\Theta(n)$ pamięci. Zatem ich złożoność jest $o(n^2)$ zgodnie z założeniami zadania.

Teraz dostając dwa wierzchołki x i y liczę odpowiadające im indeksy w tablicy `Depth`: $i_x = \text{InOrderInv}[x]$, $i_y = \text{InOrderInv}[y]$. Niech b.s.o. $i_x < i_y$ (jeśli $x = y$, to zwracam x i kończę program). Teraz będę szukać indeksu w tablicy `Depth`, w którym jest minimalna wartość głębokości na przedziale $[i_x, i_y]$. Najpierw sprawdzam, które bloki są zawarte w całości w tym przedziale dzieląc całkowicie i_x i i_y przez $\lceil \sqrt{n} \rceil$. Tych bloków jest co najwyżej $\lceil \sqrt{n} \rceil$ i biorę wszystkie wartości minimów dla tych bloków. Oprócz tego sprawdzam, na które bloki tylko zachacza dany przedział. Mogą być tylko takie dwa bloki zatem elementów zachaczonych będzie na pewno mniej niż $2\lceil \sqrt{n} \rceil$. Teraz przechodzę przez wszystkie elementy wyróżnione (w sensie przez minima w pełni zawartych bloków w przedziale i przez wszystkie elementy zachaczające o jakiś przedział nie w całości) Tych elementów jest mniej niż $3\lceil \sqrt{n} \rceil$ więc znajdowanie w nich indeksu minimalnej głębokości to złożoność czasowa $\Theta(\sqrt{n})$. Gdy indeks jest już znaleziony to przekształcam go na numer wierzchołka poprzez tablicę `InOrder` i zwracam jako wynik. Wynik zatem wykonuje się w czasie $o(n)$ zgodnie z założeniami zadania.

Wynik jest poprawny, bo gdy dostajemy x i y oraz b.s.o. x pojawia się przed y w `InOrder` i niech p to będzie ich najniższy wspólny przodek:

I p jest między x i y w `InOrder`:

x koniecznie musi być jednym z lewych (lewym i odpowiednio prawym potomkiem p nazywam wierzchołek, którego przodkiem jest p oraz jego lewe, odpowiednio prawe, dziecko) potomków p , a y jednym z prawych potomków. Bo gdyby było na odwrót to x nie pojawiłby się przed y w `InOrder` - sprzeczność. A gdyby x i y byłiby oboje lewymi (prawymi) potomkami, to rozważając poddrzewo samych lewych (prawych) potomków, znaleźlibyśmy tam inny(!) wierzchołek, który byłby w tym drzewie (i w oryginalnym naddrzewie) najniższym wspólnym przodkiem x i y , co przeczy temu, że p jest ich najniższym wspólnym przodkiem. Zatem x jest lewym potomkiem p , a y prawym. Co oznacza, że w `InOrder` najpierw zapiszemy x , potem p , a potem y .

II Pomędzy x i y są wyłącznie potomkowie p i samo p :

Wynika to ze sposobu przeglądania drzewa porządkiem `InOrder`. Rozważmy moment przeglądania od x . Jest on lewym potomkiem p , więc pomiędzy x i p znajdują się wyłącznie potomkowie p . Po p wypisywani są prawi potomkowie p , aż do momentu gdy wypisany jest y , który sam jest potomkiem p . Zatem **II** zachodzi.

III p jest tym wierzchołkiem pomiędzy x i y , który ma najmniejszą głębokość.

Oznaczmy wierzchołek pojawiający się pomiędzy x i y w porządku `InOrder`, który ma najmniejszą głębokość jako q . Po pierwsze istnieje tylko jedno takie q , bo jeśli istniałyby dwa wierzchołki o tej samej głębokości, to pomiędzy nimi musiałyby być jakiś wierzchołek mniejszej głębokości bo pomiędzy nimi musiałyby się pojawić na przykład rodzic któregoś z nich, bo braci zawsze oddziela rodzic w `InOrder`, co prowadzi do sprzeczności. Zatem q jest dobrze określone. Ale z **II** wynika, że albo $q = p$, albo q jest potomkiem p , ale gdyby tak było to głębokość p byłaby mniejsza od głębokości q - sprzeczność. Zatem $q = p$.

To rozumowanie uzasadnia poprawność algorytmu.